

Basic Building Blocks

Lecture 5

Dr. Shoab A. Khan

Adders

- Used in addition, subtraction, multiplication and division
- Speed of a signal processing or communication system ASIC depends heavily on these functional units

Half Adder using Data Flow modeling

```
module HALF_ADDER(ai, bi, si, cout);
```

```
    input ai, bi;
```

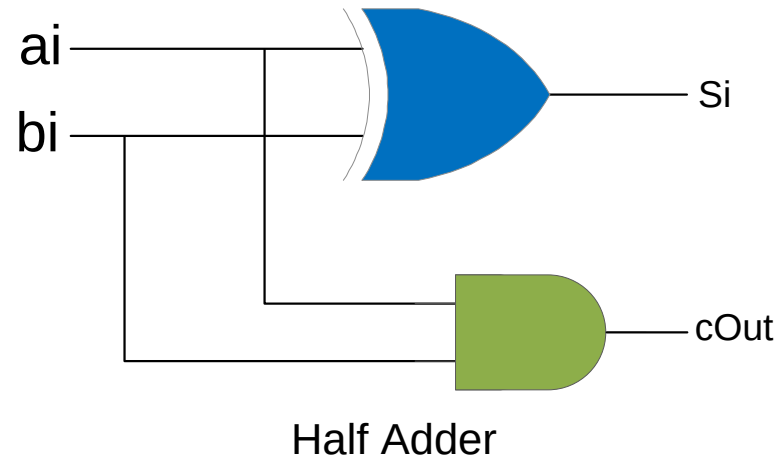
```
    output si, cout;
```

```
    // data flow modeling
```

```
    assign {cout,si} = ai + bi;
```

```
endmodule
```

Half Adder using Data Flow modeling



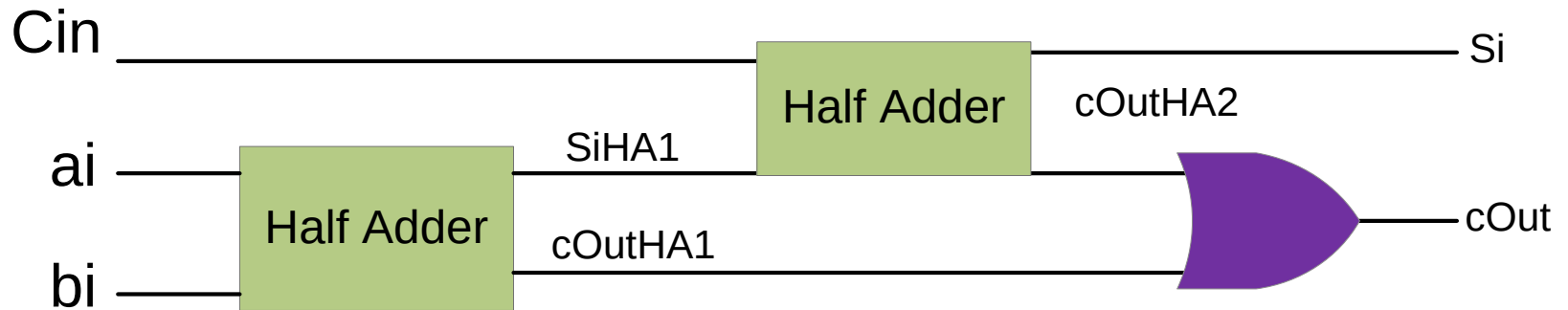
$$C_i = a_i b_i$$

$$S_i = a_i \oplus b_i$$

Full Adder

Truth Table				
x	y	z	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

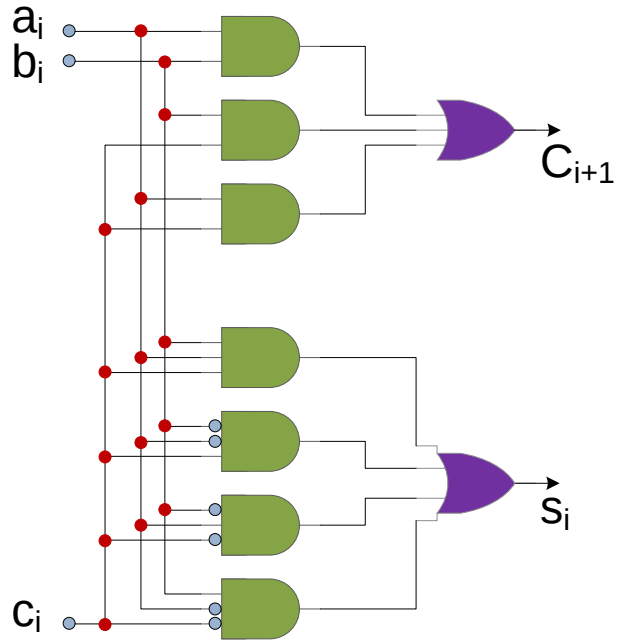
Full Adder



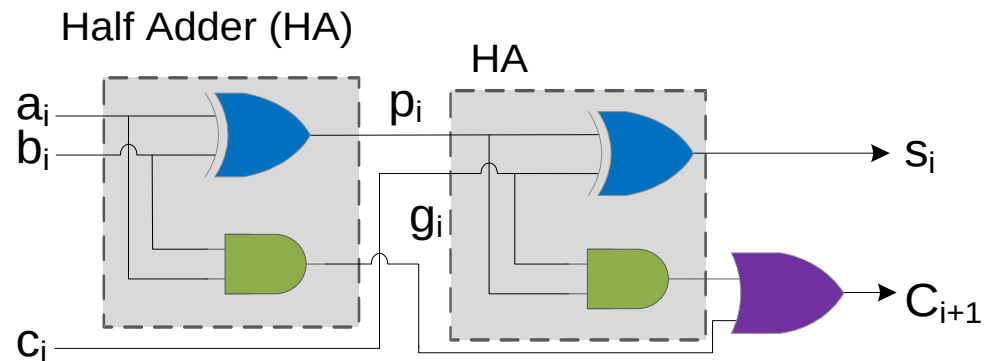
$$c_i = (a_i \oplus b_i) c_{i-1} + a_i b_i$$

$$s_i = a_i \oplus b_i \oplus c_i$$

Gate-level design options for a full adder

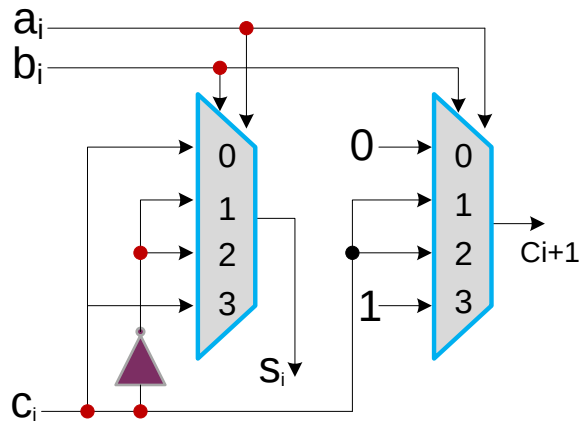


(a)

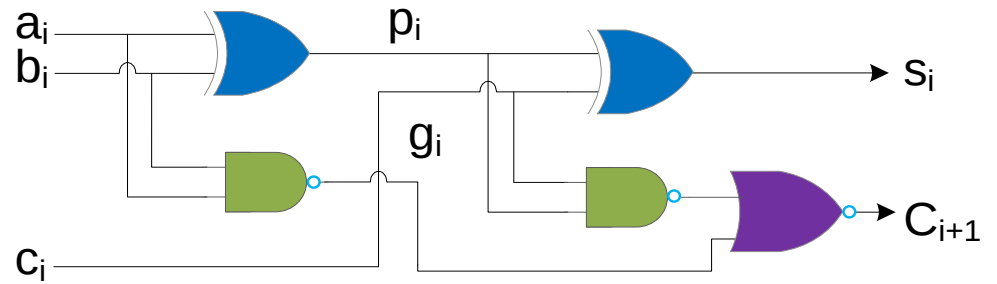


(b)

Contd ...



(c)



(d)

Full Adder: Implementation in Verilog

```
module FULL_ADDER(ai,bi,cin,si,cout);  
  
    input ai,bi;  
    input cin;  
    output si,cout;  
    wire SiHA1,CoutHA1,CoutHA2;  
  
    HALF_ADDER HA1(ai,bi,SiHA1,CoutHA1); // instance HA1  
    HALF_ADDER HA2(SiHA1,cin,si,Cout); //instance HA2  
    Or (cout,CoutHA1,CoutHA2); // using or gate primitive  
  
endmodule
```

Full Adder Using Data Flow Modeling

```
module FULL_ADDER(ai,bi,cin,si,cout);
```

```
    input ai,bi;
```

```
    input cin;
```

```
    output si,cout;
```

```
// through data flow level of abstraction
```

```
assign {cout,si} = ai + bi + cin;
```

```
endmodule
```

Full Adder Using Data Flow Modeling

```
module FULL_ADDER(ai,bi,cin,si,cout);
```

```
    input ai,bi;
```

```
    input cin;
```

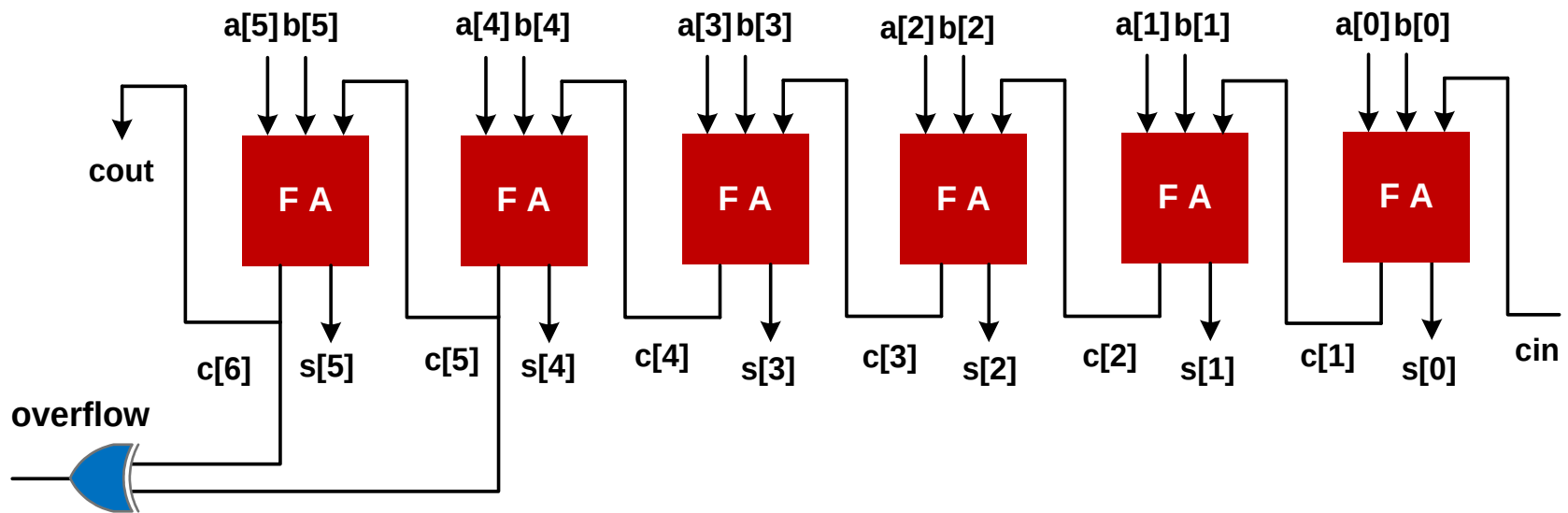
```
    output si,cout;
```

```
    // through data flow level of abstraction
```

```
    assign {cout,si} = ai + bi + cin;
```

```
endmodule
```

Ripple Carry Adder



```

module ripple_carry_adder #(parameter W=16)
    (input      clk,
     input  [W-1:0] a, b,
     input      cin,
     output reg [W-1:0] s_r,
     output reg      cout_r);

    wire [W-1:0] s;
    wire      cout;
    reg [W-1:0] a_r, b_r;
    reg      cin_r;

    assign {cout,s} = a_r + b_r + cin_r;

    always@(posedge clk)
    begin
        a_r<=a;
        b_r<=b;
        cin_r<=cin;
        s_r<=s;
        cout_r<= cout;
    end
endmodule

```

RCA: Dataflow modeling

Six bit ripple carry adder through data flow modeling

```
// SIX BIT FULL ADDER ;  
module fulladder_6bit(s,cout,a,b,cin);  
  
output cout;  
output [5:0] s;  
input [5:0] a,b;  
input cin;  
reg [5:0] s,c;  
reg cout;  
always@(a or b or cin)  
begin  
    {c[0],s[0]}= a[0] + b[0] + cin;  
    for(i=1; i<6; i=i+1)  
        {c[i],s[i]}= a[i] + b[i] + c[i-1]; // through data flow modeling.  
    cout = c[5];  
end  
endmodule
```

Important Observation

- Do we have to wait for the carry to show up to begin doing useful work?
 - ❑ We do have to know the carry to get the right answer.
 - ❑ But, it can only take on two values

Non-uniform Group 12-Bit Carry Select Adder

- Three partitions of 3-bits, 4-bits, 5-bits are made
- The cout of the first block is ready earlier making it faster in functionality than the uniform group 12- bit carry select adder
- So non-uniform group carry select adder is faster than the uniform group carry select adder

Carry Generate and Propagate Logic

$$g_i = a_i b_i$$

$$p_i = a_i \oplus b_i$$

$$c_{i+1} = g_i + p_i c_i$$

$$s_i = c_i \oplus p_i$$

Group Carry and Group Propagate

$$c_1 = g_0 + p_0 c_0$$

$$c_2 = g_1 + p_1 c_1$$

$$= g_1 + p_1 g_0 + p_1 p_0 c_0$$

$$= g_1 + p_1 g_0 + p_0 p_1 c_0$$

$$c_3 = g_2 + p_2 g_1 + p_2 p_1 g_0 + p_2 p_1 p_0 c_0$$

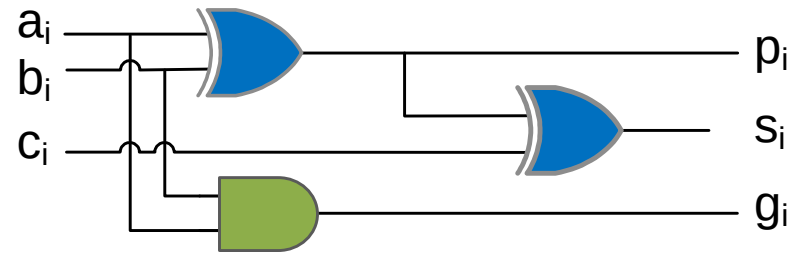
$$c_4 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0 + p_3 p_2 p_1 p_0 c_0$$

$$\text{let } G_0 = g_3 + p_3 g_2 + p_3 p_2 g_1 + p_3 p_2 p_1 g_0$$

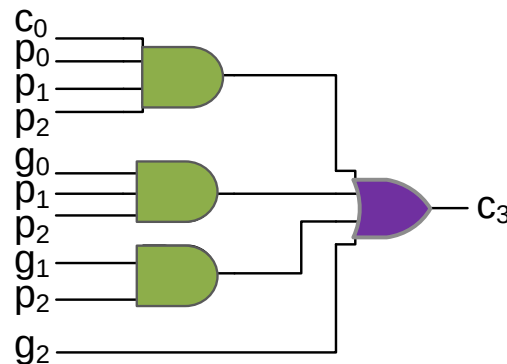
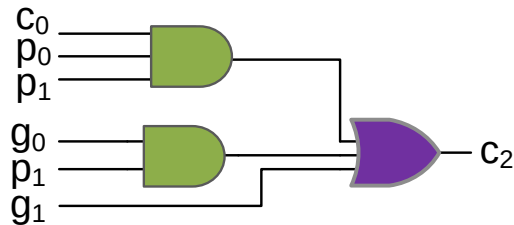
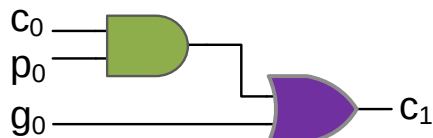
$$\text{and } P_0 = p_3 p_2 p_1 p_0$$

$$\text{we can write } c_4 = G_0 + P_0 c_0$$

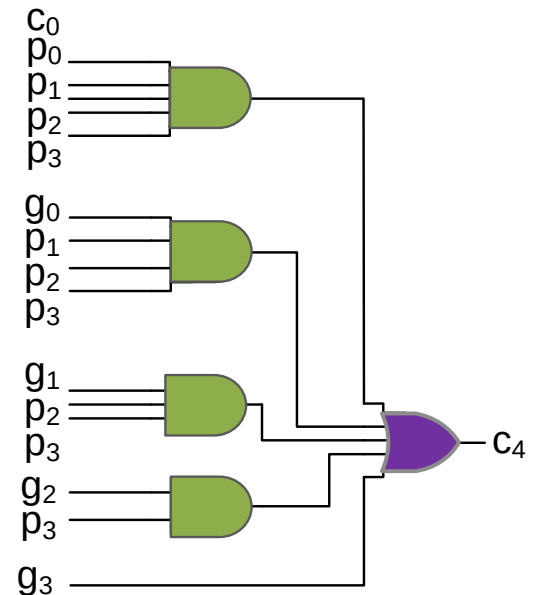
CLA logic for computing carries in two-Gate delay time



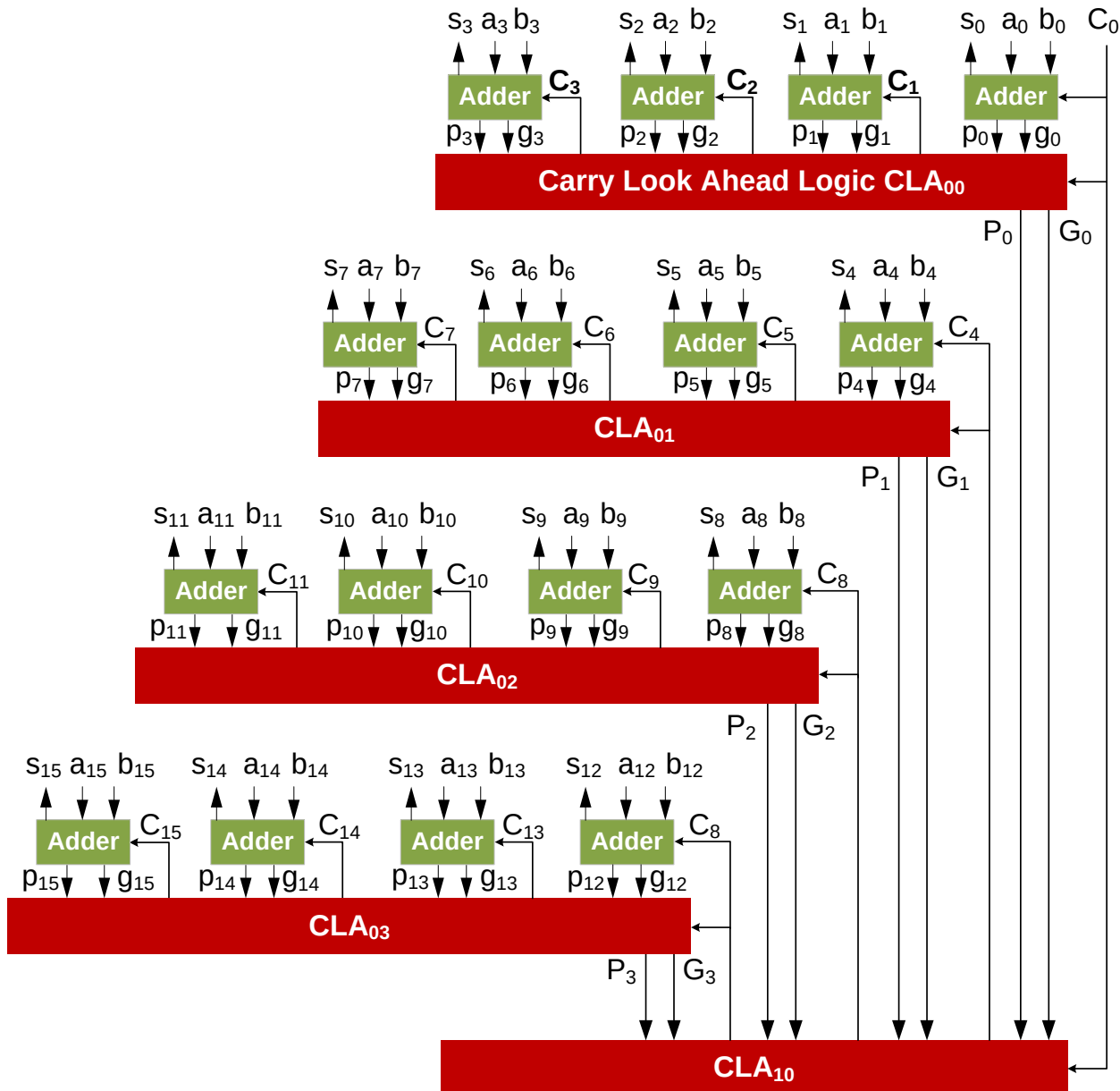
(a)



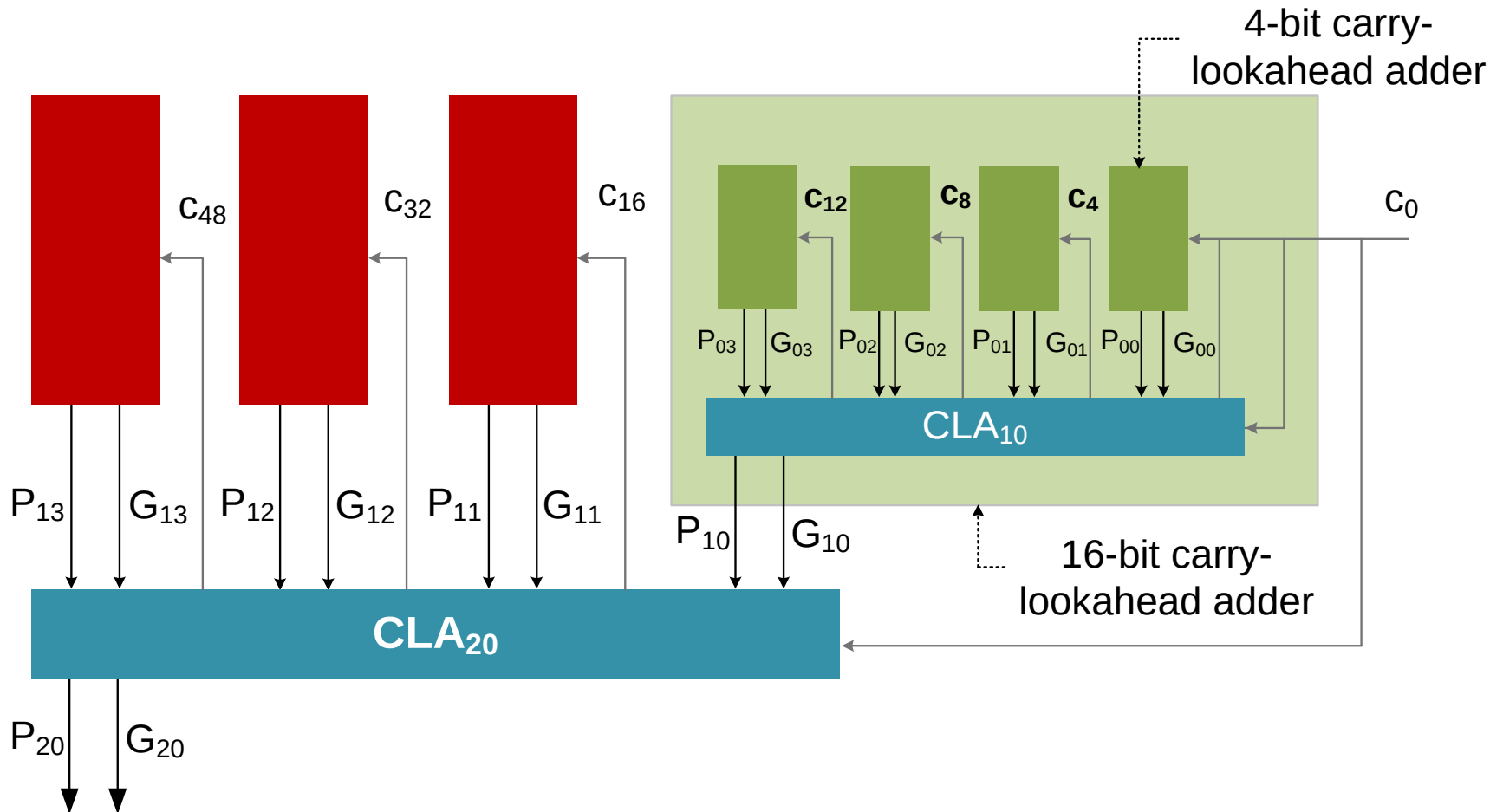
(b)



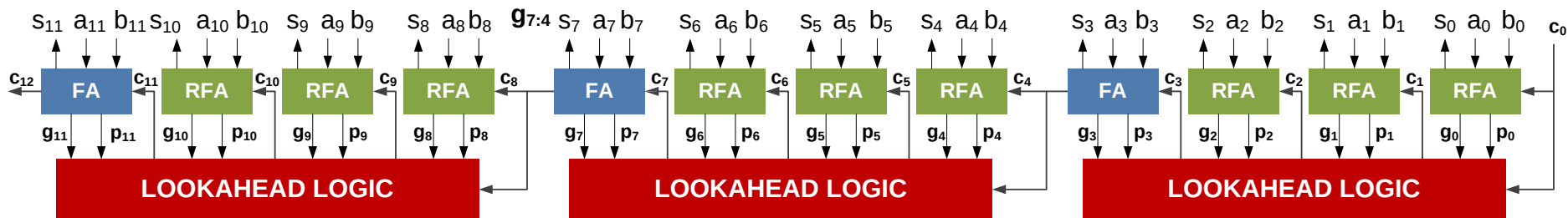
A 16-bit carry look-ahead adder using two levels of CLA logic



A 64-bit carry look-ahead adder using three levels of CLA logic



A 12-bit Hybrid Ripple Carry and Carry Look-ahead Adder



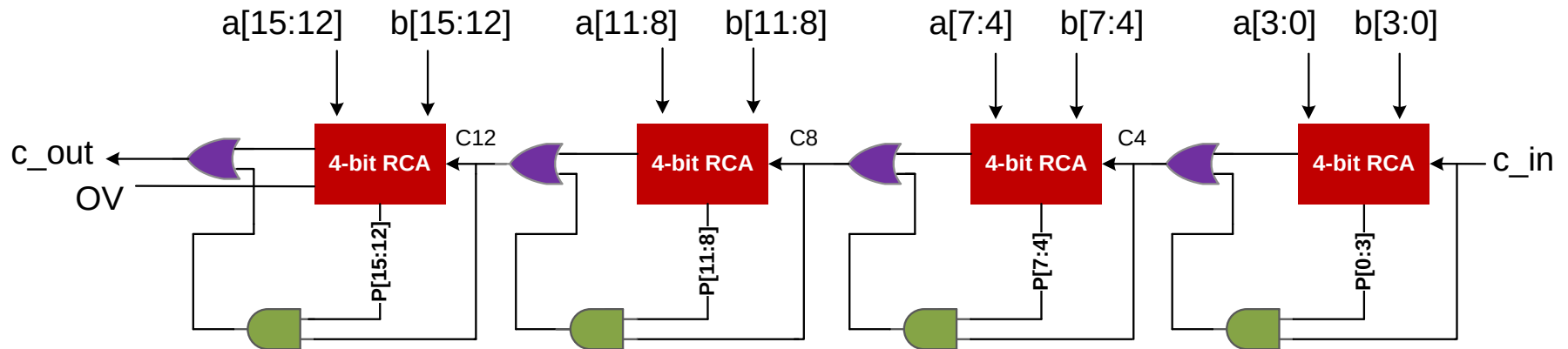
Carry Skip Adder

- If any group generates a carry, it passes it to the next group
- In case the group does not generate its own carry then it simply bypasses the carry from the previous block to its next block

$$P_i = p_i p_i + 1p_i + 2...p_i + k - 1$$

$$p_i = a_i \oplus b_i$$

A 16-bit equal-group carry skip adder



Conditional Sum Adder

- The process that led to the two-level carry select adder can be continued . . .
- A logarithmic time *conditional-sum adder* results if we proceed to the extreme:
 - single bit adders at the top
- A conditional-sum adder is actually a $(\log_2 k)$ -level carry-select adder
- Implemented in multiple levels
- Built using Conditional Cells (CC) and MUX(s)

Principle

- The conditional cell generates a pair of sum and carry bits i at each bit position ($si0, ci0, si1, ci1$)
- One pair assumes carry_in of one ($si1, ci1$) and the other assumes a carry_in of zero ($si0, ci0$)
- The correct sums and carries are then selected using a tree of multiplexers
- All level one bits are paired up
- The sum and carry of the next bit position, brought down to level 2 are selected by the least significant carry
- This continues until all the sums and carries are resolved in the last level

Example

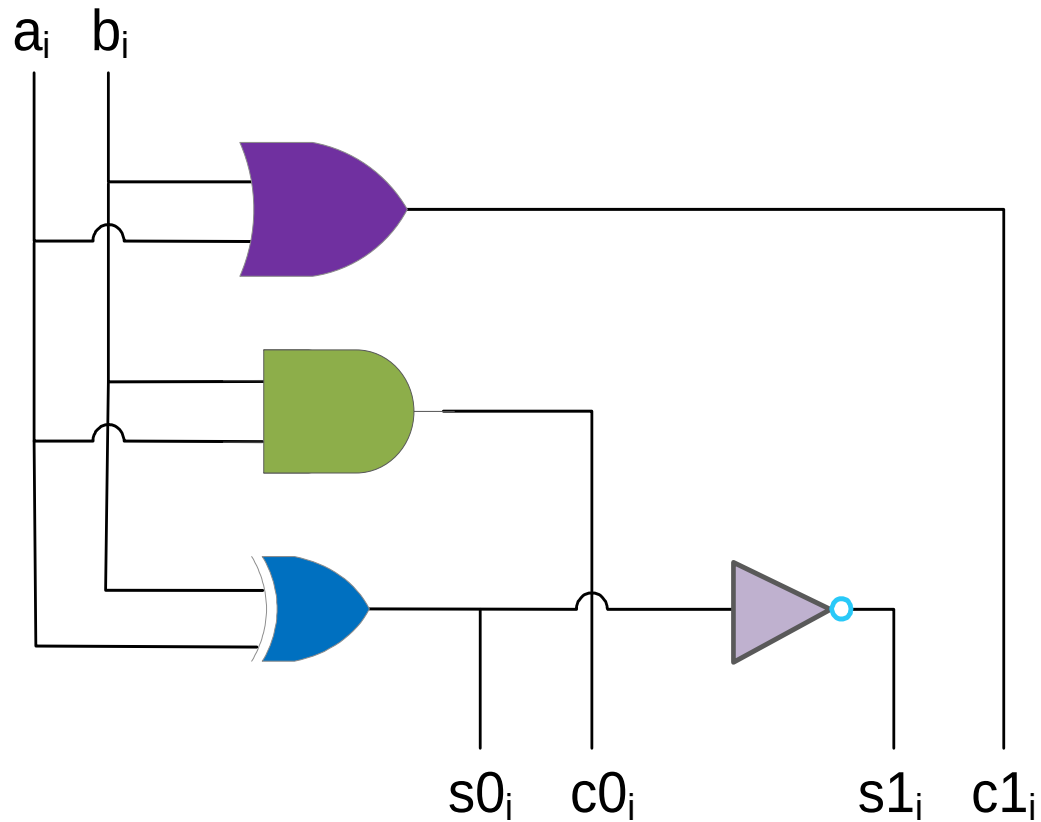
$$s0_i = a_i \oplus b_i$$

$$s1_i = a_i \sim \oplus b_i$$

$$c0_i = a_i b_i$$

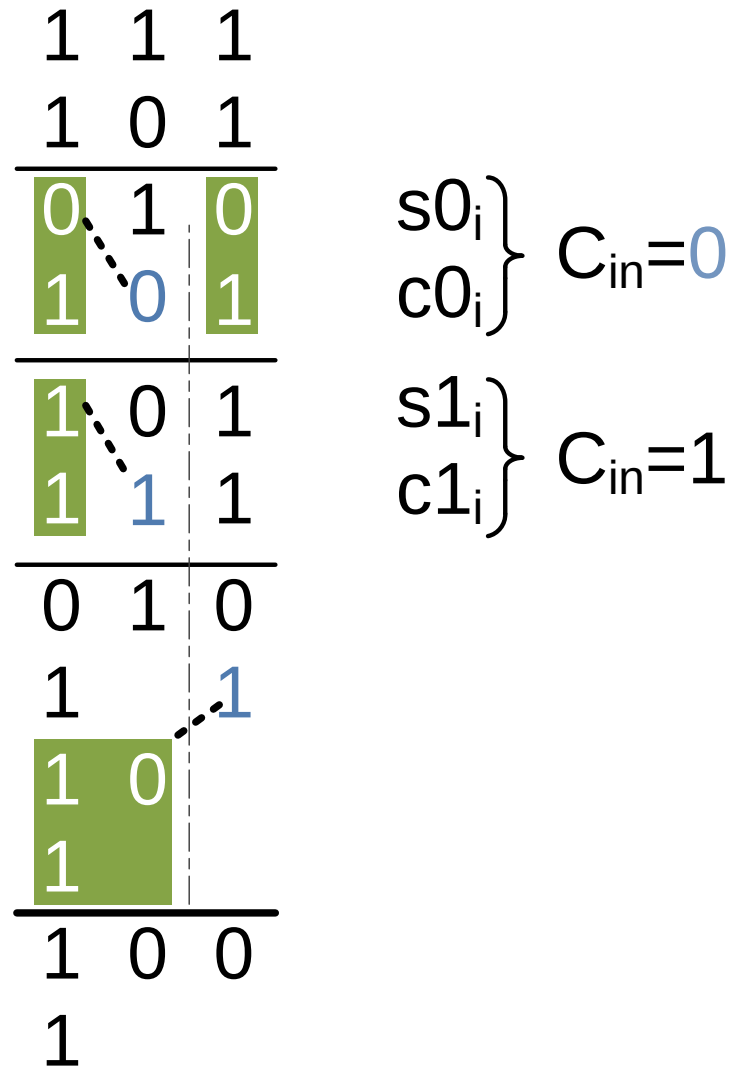
$$c1_i = a_i + b_i$$

Conditional Cell (CC)



Addition of three bit numbers using a conditional sum adder

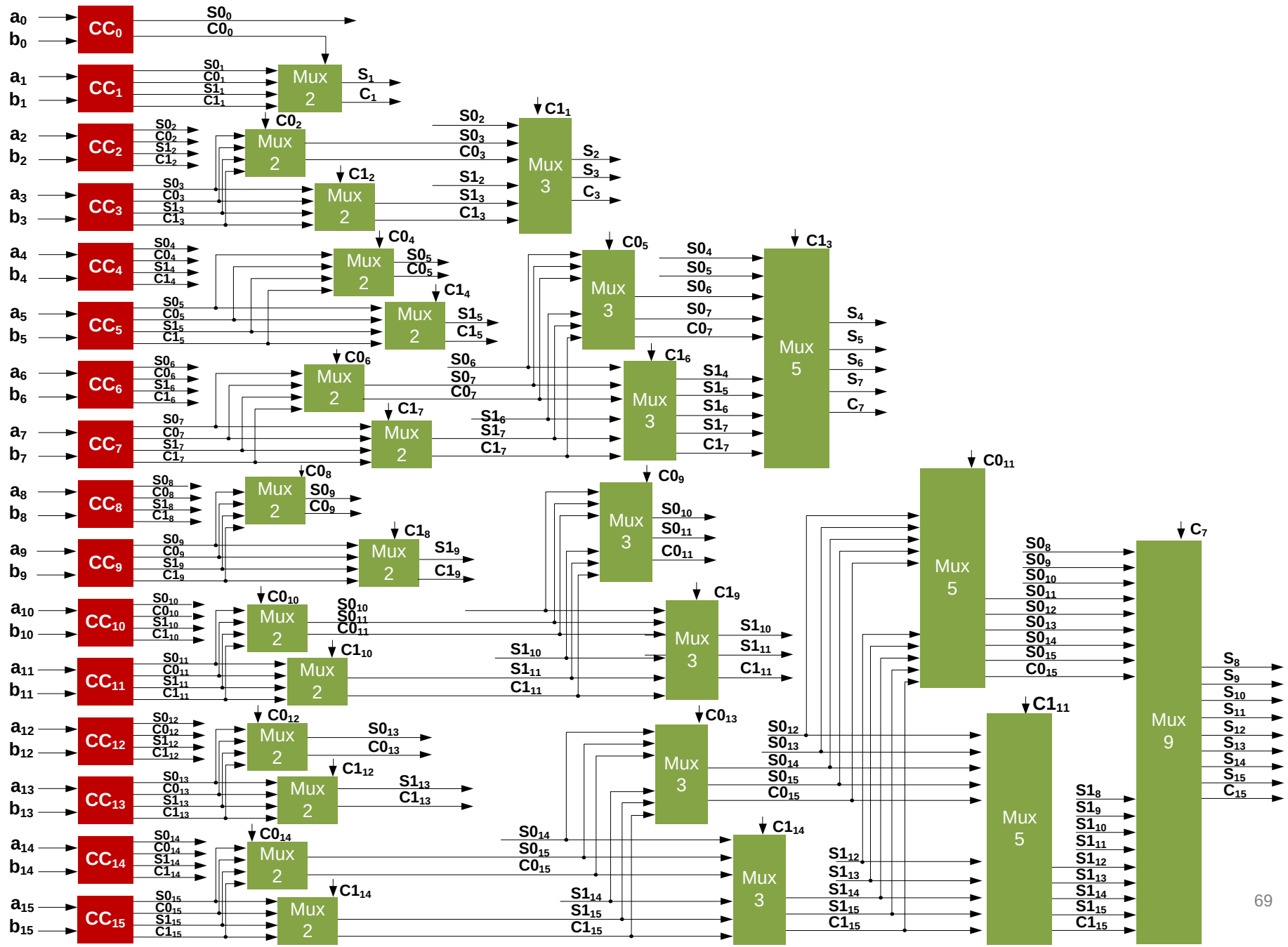
(Here we are assuming actual $c_{in}=0$)



Example: Conditional Sum Adder

		a_i	1	0	0	1	1	0	0	1	1	1	0	0	1	1	0	1	a_i
		b_i	0	0	1	1	0	1	1	0	1	1	0	1	0	1	1	0	b_i
Group width	Group carry-in	Group sum and block carry out																	i
		15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
1	0	1	0	1	0	1	1	1	1	0	0	0	1	1	0	1	1	$S0_i$	
		0	0	0	1	0	0	0	0	1	1	0	0	0	1	0	0	$C0_i$	
	1	0	1	0	1	0	0	0	0	1	1	1	0	0	1	0		$S1_i$	
		1	0	1	1	1	1	1	1	1	1	0	1	1	1	1		$C1_i$	
2	0	1	0	0	0	1	1	1	1	1	0	0	1	0	0	1	1		
		0		1		0		0		1		0		1		0			
	1	1	1	0	1	0	0	0	0	1	1	1	0	0	1				
		0		1		1		1		1		0		1					
4	0	1	1	0	0	1	1	1	1	1	0	0	1	0	0	1	1		
		0				0				1				1					
	1	1	1	0	1	0	0	0	0	1	0	1	0						
		0				1				1									
8	0	1	1	0	0	1	1	1	1	1	0	1	0	0	0	1	1		
		0								1									
	1	1	1	0	1	0	0	0	0										
		0																	
16	0	1	1	0	1	0	0	0	0	1	0	1	0	0	0	1	1		
		0																	
	1																		

A 16-bit Conditional Sum Adder



Hybrid Adder Designed

- Hybrids are obtained by combining elements of:
 - Ripple-carry adders
 - Carry-lookahead (generate-propagate) adders
 - Carry-skip adders
 - Carry-select adders
 - Conditional-sum adders

- You can obtain adders with
 - higher performance
 - greater cost-effectiveness
 - lower power consumption

Example

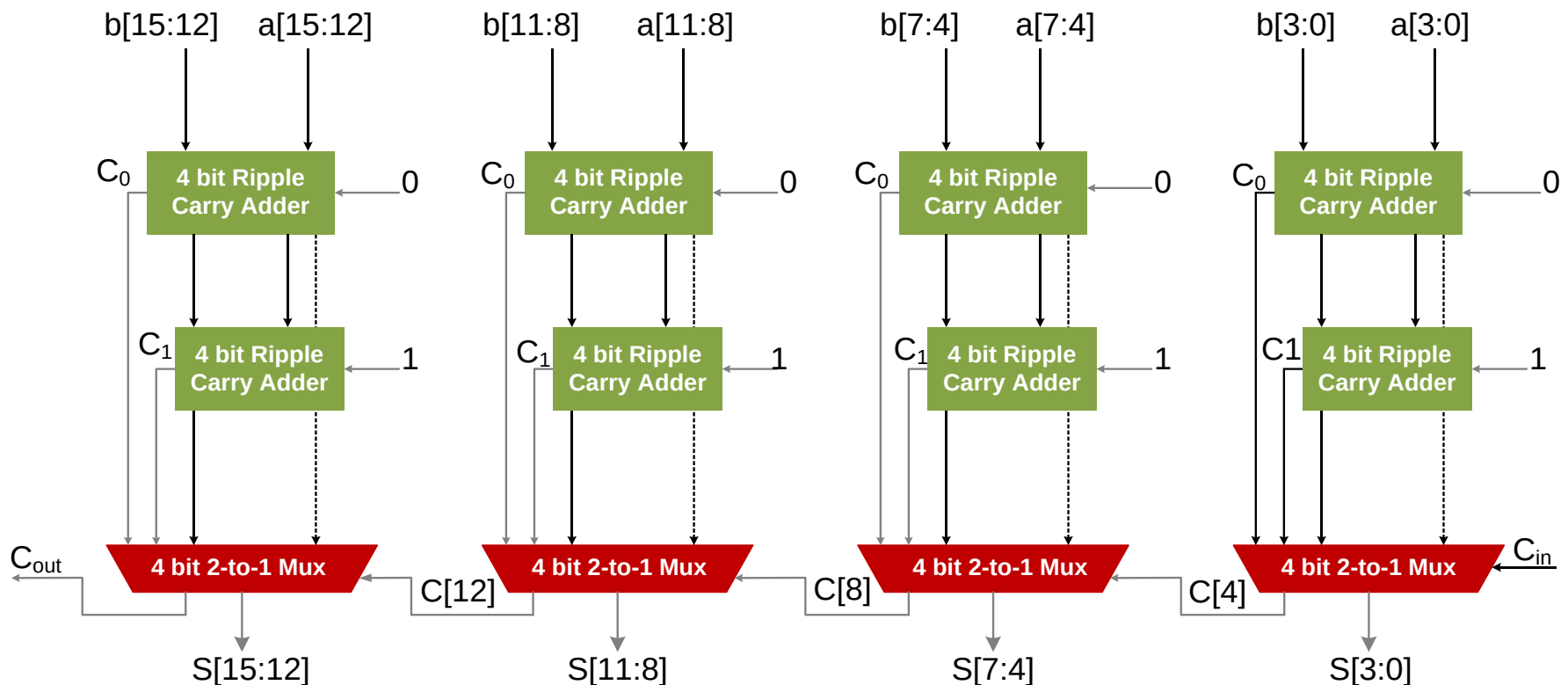
$$s0_i = a_i \oplus b_i$$

$$s1_i = a_i \sim \oplus b_i$$

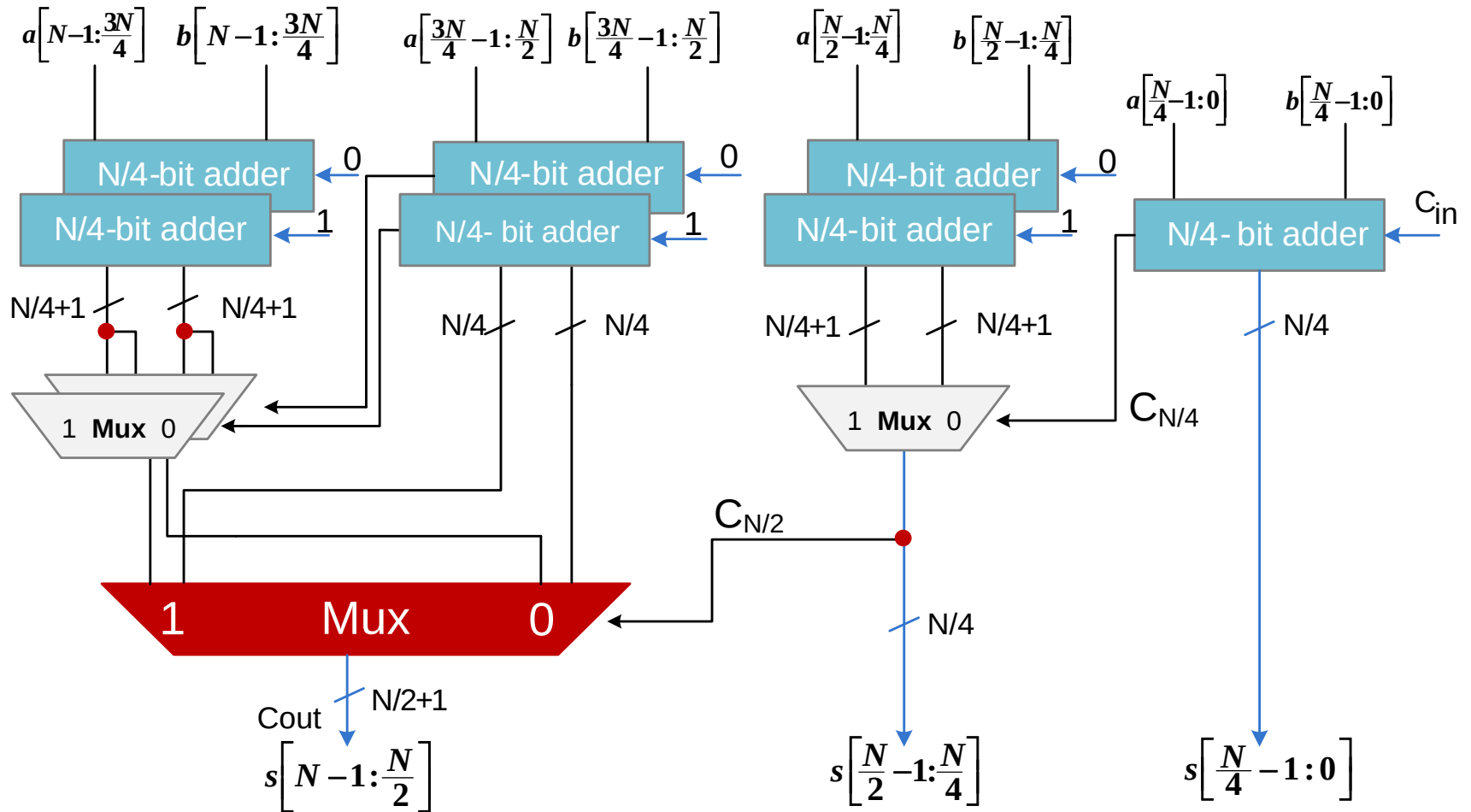
$$c0_i = a_i b_i$$

$$c1_i = a_i + b_i$$

A 16 –bit uniform-groups carry select adder



Hierarchical CSA

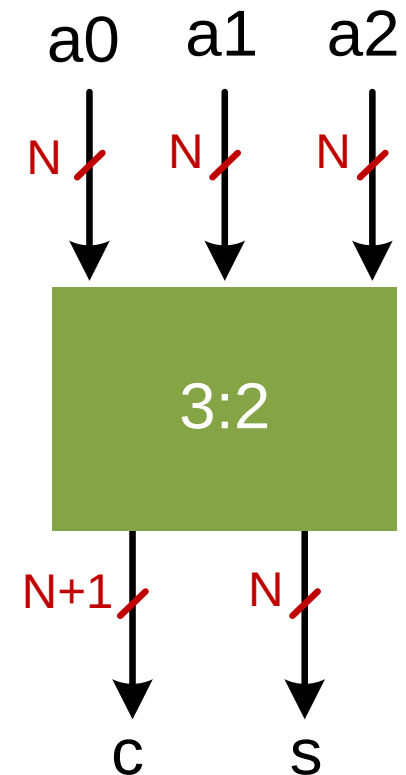


Carry Save Adders and Compressors

Carry Save Addition saves the carry at next bit location

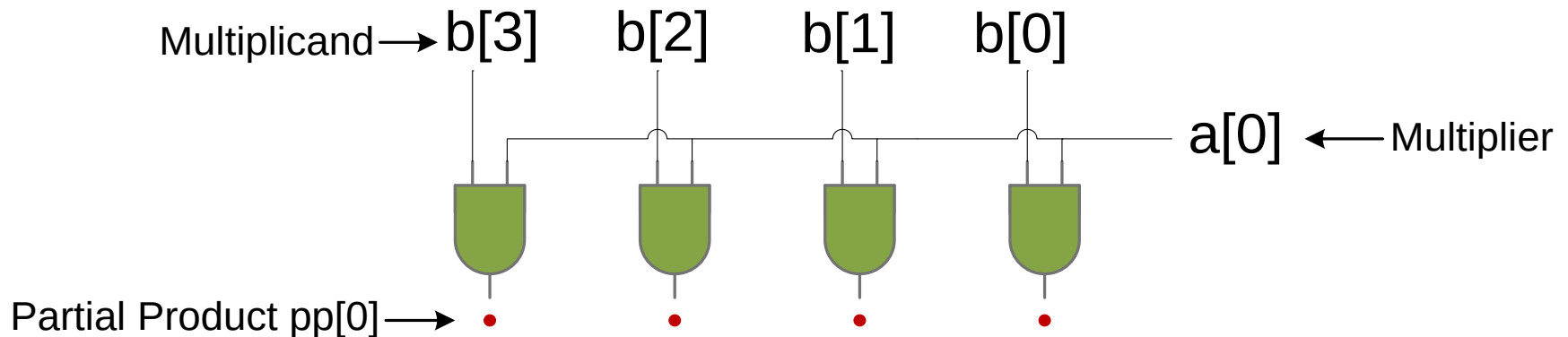
- The CSA does not ripple any carry
- It has a delay of one FA
- The concept of CSA is effective in designing partial products compression/reduction logic

a0	=		0	0	1	0	1	1
a1	=		0	1	0	1	0	1
a2	=		1	1	1	1	0	1
s	=		1	0	0	0	1	1
c	=	0	1	1	1	0	1	



Dots are used to represent each bit of the partial product

- Dot notation facilitates description of different reduction schemes
- Dots are used to represent each bit of the partial product



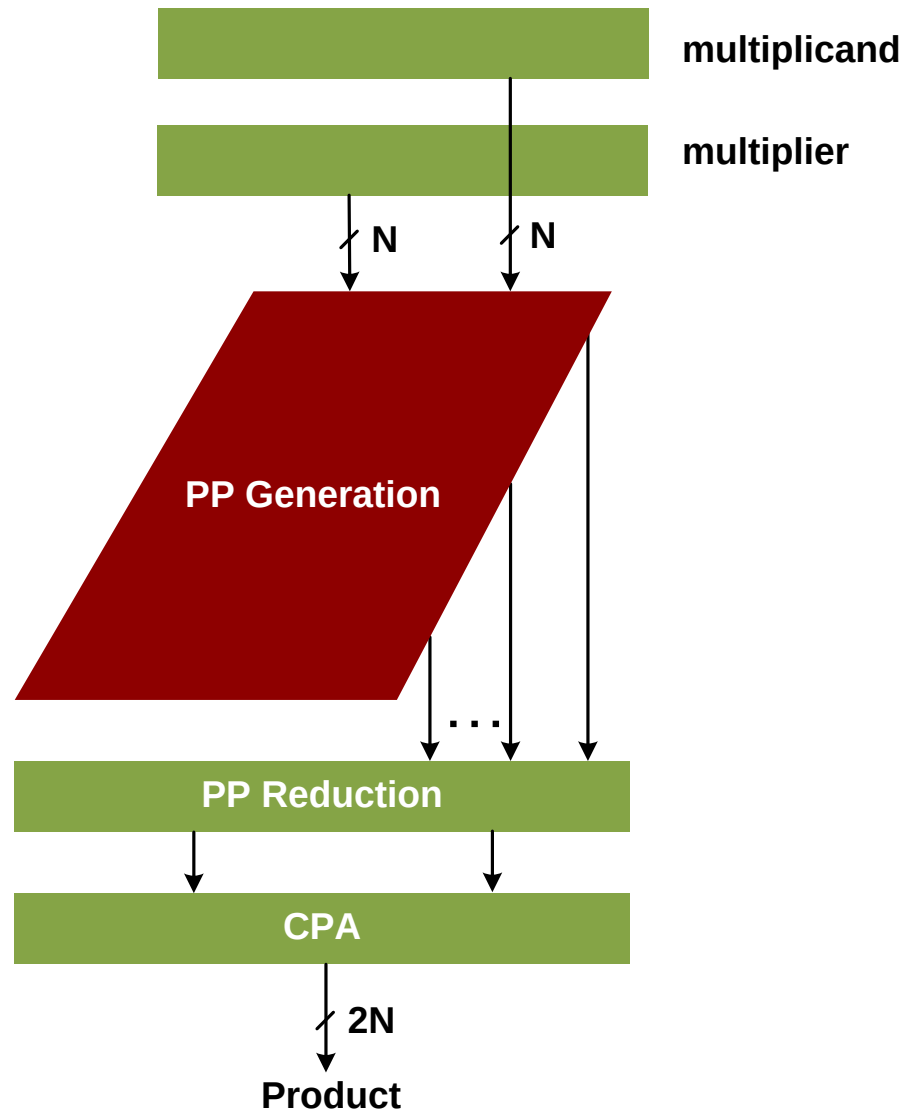
Parallel multiplier architecture

Designing Customized Multipliers

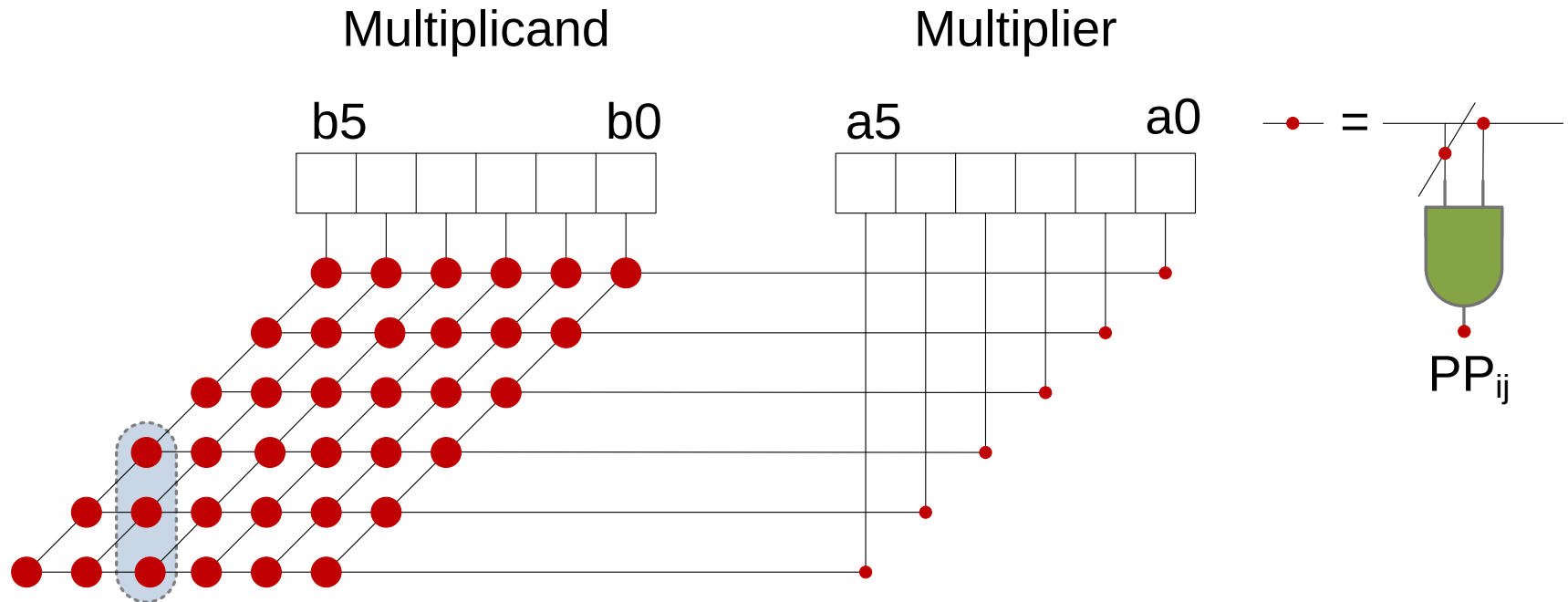
Three components of a multiplier

- N-bit inputs operands
- Partial Product Array Generation = N shifted binary numbers
- Partial Product Array Reduction= reduction to 2 binary numbers
- Final addition = 2N-bit final product

Three components of a multiplier



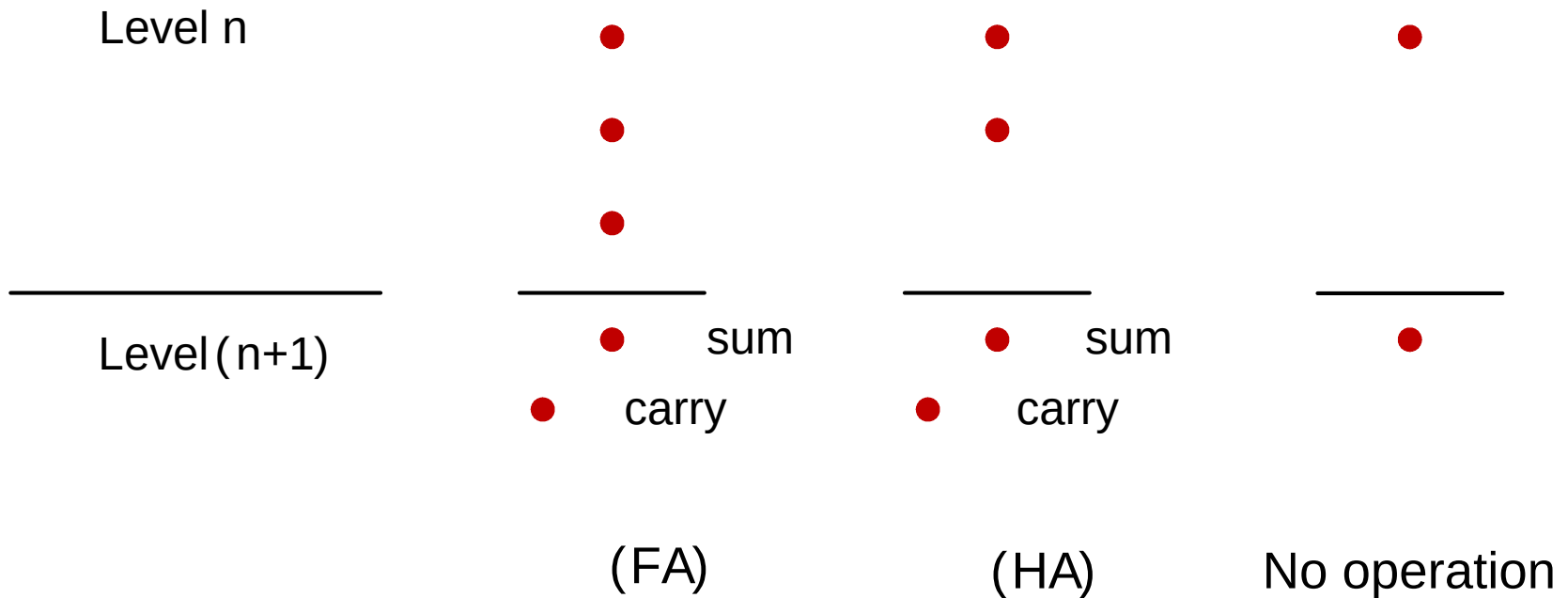
Partial Product Generation for a 6x6 Multiplier



Partial Product Generation Verilog Code

```
module multiplier (  
    input [5:0] a,b,  
    output [11:0] prod);  
  
    integer i;  
  
    reg [5:0] pp [0:5]; // 6 partial products  
  
    always@*  
    begin  
        for(i=0; i<6; i=i+1)  
        begin  
            pp[i] = b & {6{a[i]}};  
        end  
    end  
  
    assign prod = pp[0]+{pp[1],1'b0}+{pp[2],2'b0}  
        +{pp[3],3'b0}+{pp[4],4'b0}+{pp[5],5'b0};  
  
endmodule
```

Reducing number of dots in a column



- Three dots are shown
- Each symbolizes a partial product
- Using FA reduces these to two bits
- One has the weight of 2^0 (sum)
- The other has the weight of 2^1 (carry)
- This type of reduction is known as 3 to 2 reduction or carry saves reduction
- The two dots are reduced to 2 using a HA

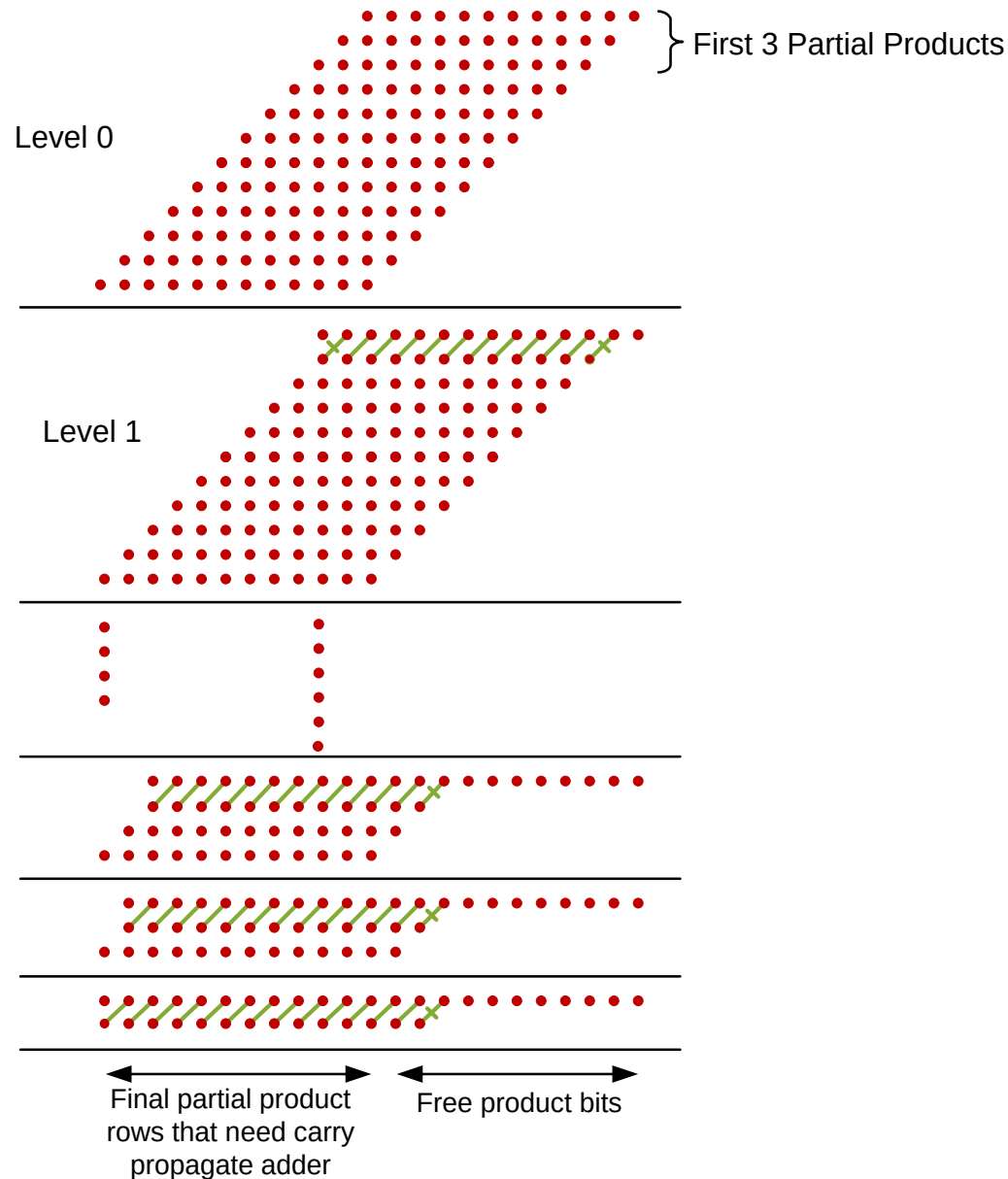
Partial Products Reduction Schemes

- Carry Save Reduction Scheme
- Dual Carry Save Reduction Scheme
- Wallace Tree Reduction Scheme
- Dadda Tree Reduction Scheme

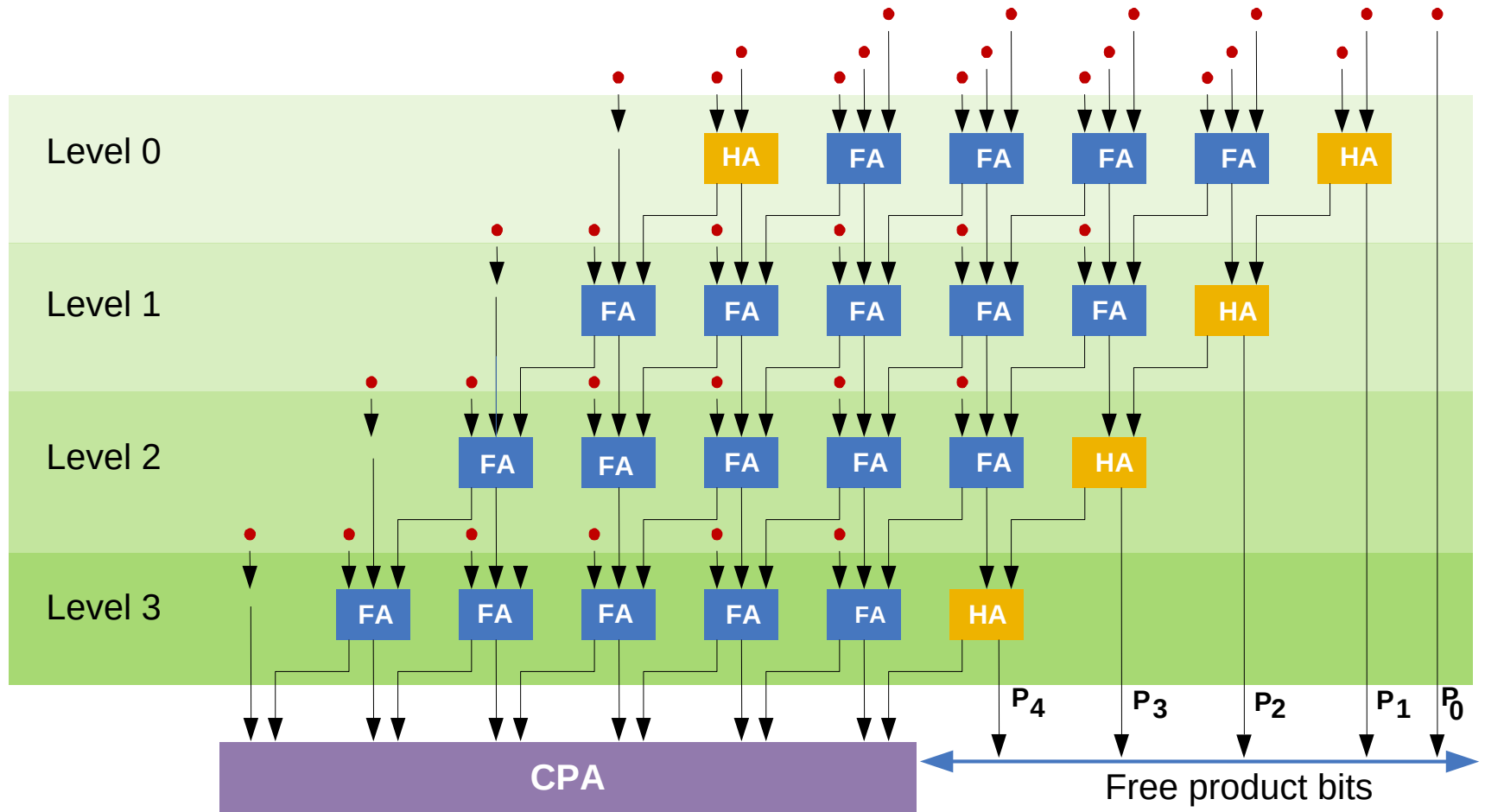
12x12 Carry Save Reduction Scheme

- Considers three rows at a time
- Take first three rows use CSA to reduce them to two
- Iteratively take two layers from previous reduction and a new from PP layer and reduce them to two using a CSA
- Finally produces two layers
- Also produces free product bits
- The two layers are added using any CPA

PP reduction for a 12x12 Multiplier using Carry Save Reduction Scheme



Carry Save Reduction Scheme Layout for a 6x6 Multiplier



Dual Carry Save Reduction

- The partial products are divided into 2 equal size groups
- The carry save reduction scheme is applied on both the groups simultaneously
- This results into two partial product layers in each group
- The four layers are then reduced using Carry Save Reduction
- The last two layers are added using any CPA

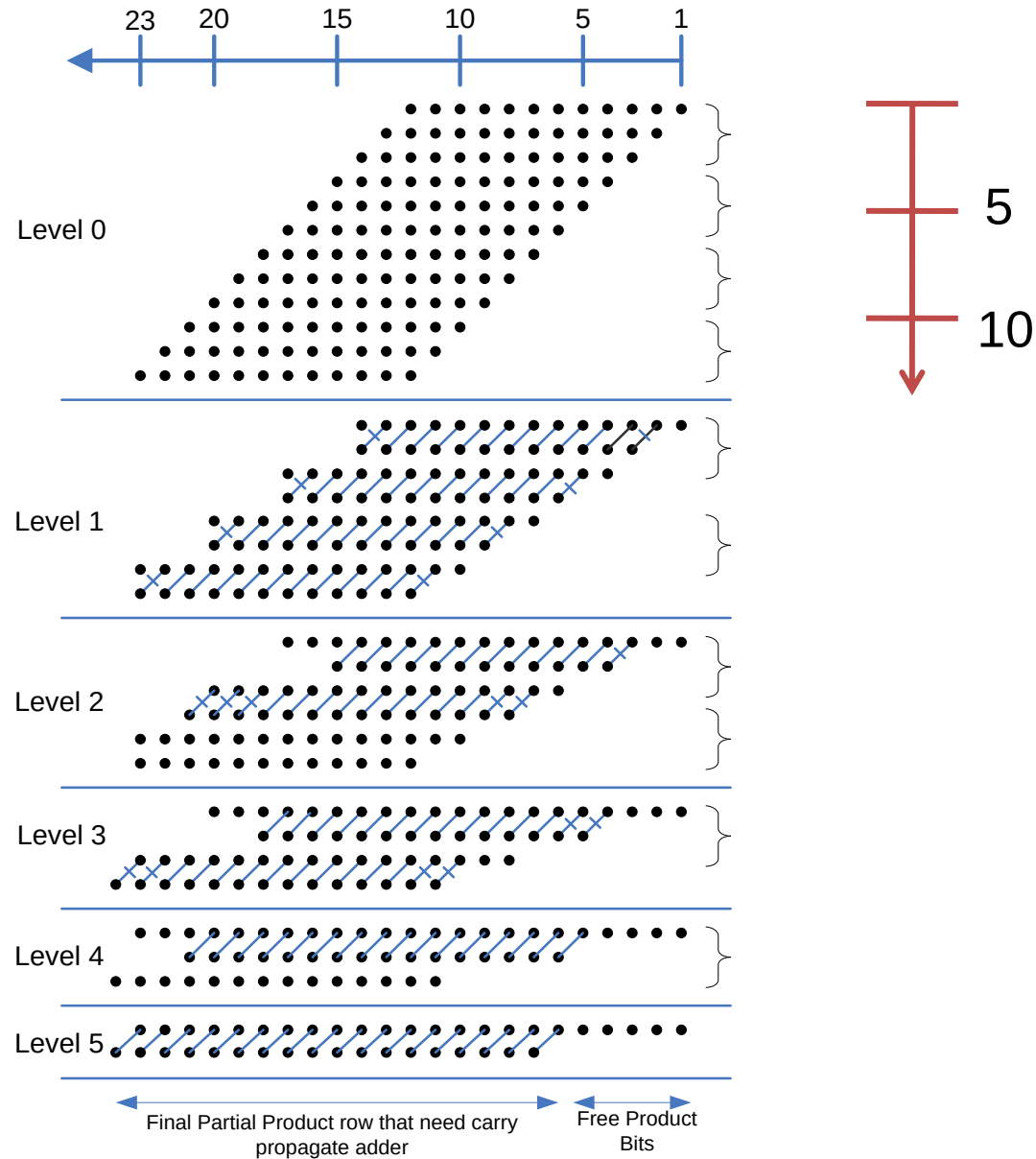
Wallace Tree Multipliers

- One of the most commonly used multiplier architecture
- It is log time array multiplier
- The number of adder levels increases logarithmically as the partial product rows increase

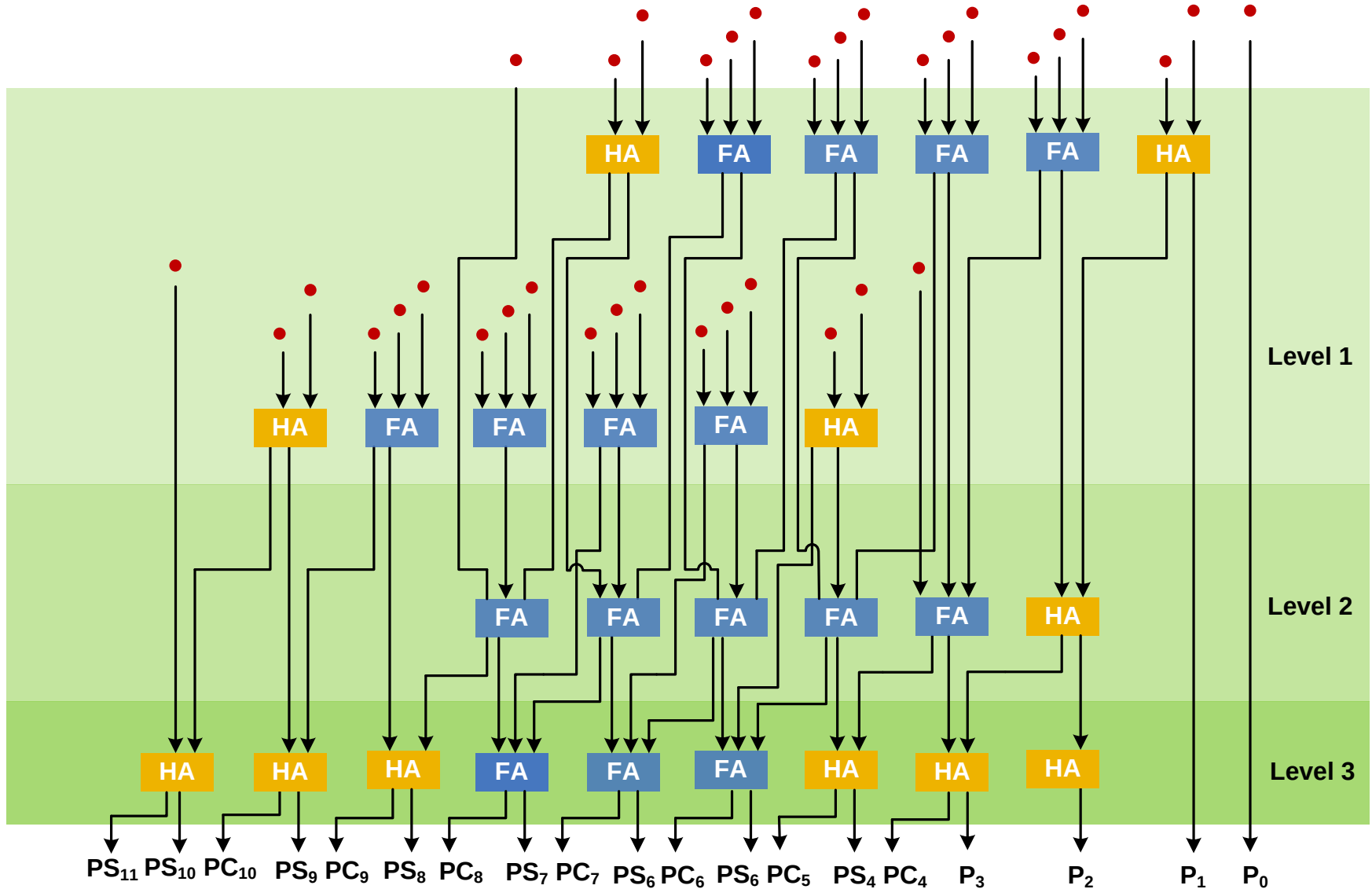
Wallace Tree Multipliers

- Make group of threes and apply CSA reduction in parallel
- Each CSA layer produces two rows
- These rows then, with other rows from other partial product groups, form a new reduced matrix
- Iteratively apply Wallace reduction on the new generated matrix
- This process continues until only two rows are left
- The final rows are added together for the final product

Wallace Reduction Tree applied on 12 PPs



Wallace Reduction layout for a 6x6 array of PPs



Dada Reduction uses the Wallace Reduction Table

Adder Levels in Wallace Tree Reduction Scheme

**Number of partial
Products**

**Number of full adder
Levels**

3

1

4

2

$5 \leq n \leq 6$

3

$7 \leq n \leq 9$

4

$10 \leq n \leq 13$

5

$14 \leq n \leq 19$

6

$20 \leq n \leq 28$

7

$29 \leq n \leq 42$

8

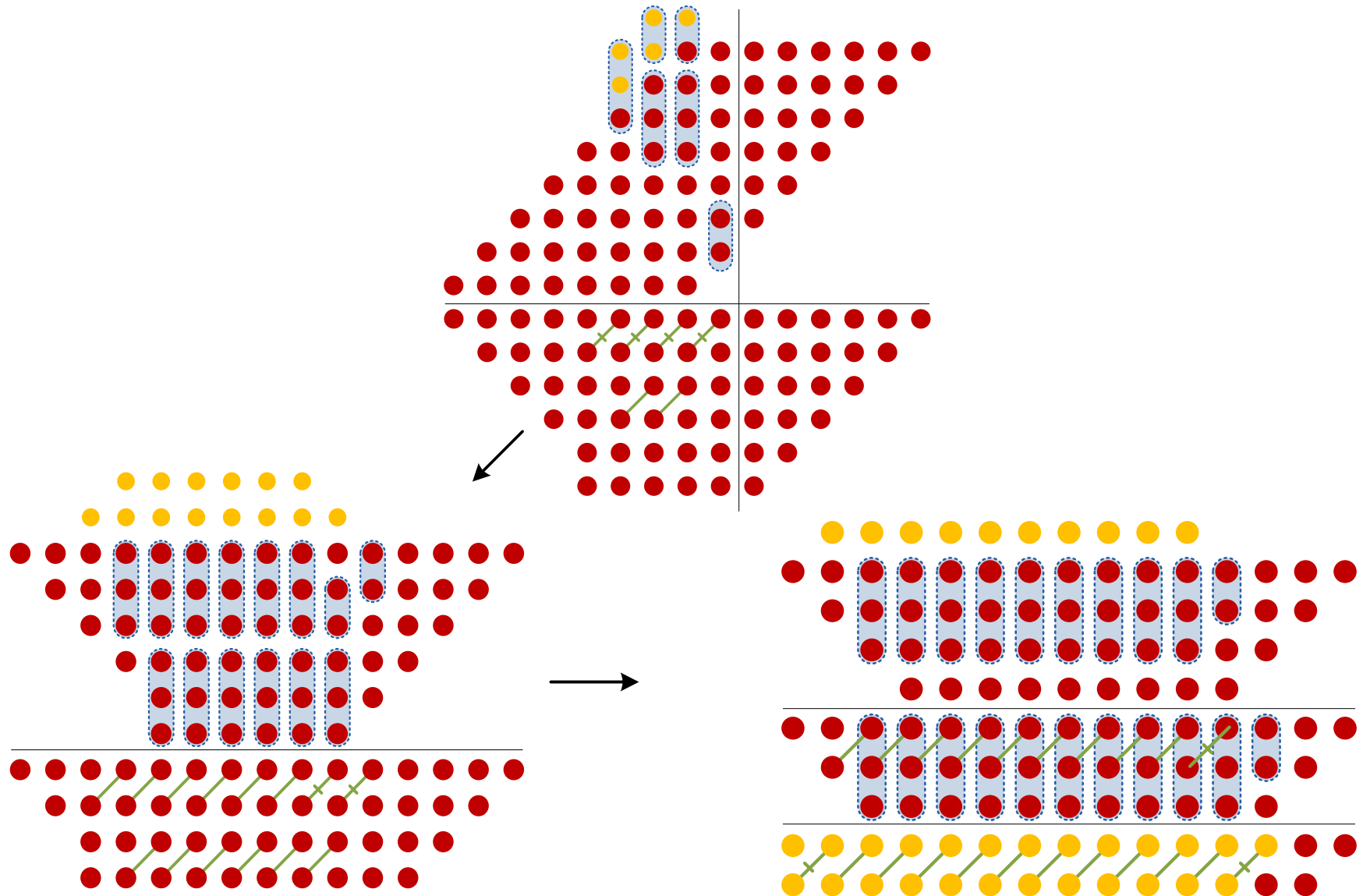
$43 \leq n \leq 63$

9

Dada Reduction

- Minimizes the number of HAs and FAs
- Reduction considers each column separately
- Reduces the number of dots in each column to the maximum number of layers in the next level in Wallace Reduction Table

Dadda reduction levels for reducing eight PPs to two



As the string property is applied on three bits, there are following eight possibilities:

$2^1=2$	$2^0=1$		
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	2
1	0	0	-2
1	0	1	-1
1	1	0	-1
1	1	1	0